

Sprawozdanie z badań: Porównanie architektur CNN w zadaniu klasyfikacji obrazów

Autor Sprawozdania

13 maja 2026

Spis treści

1	Cel i zakres zadania	2
2	Opis środowiska i narzędzi	2
3	Organizacja badań	2
4	Opis zbioru danych i przygotowanie	2
5	Opis modeli	3
6	Miary oceny jakości modeli	3
6.1	Oznaczenia	3
7	Wyniki	4
7.1	Najlepsze wyniki dla każdego z modeli	5
7.2	Najsłabsze wyniki dla każdego z modeli	5
7.3	Macierze pomyłek	5
7.4	Analiza krzywych ROC i wartości AUC	9
7.5	Wizualizacja predykcji dla najlepszych konfiguracji	10
8	Walidacja modeli i analiza przeuczenia	17
8.1	Analiza wyników na zbiorze walidacyjnym	17
8.2	Macierze pomyłek dla zbioru walidacyjnego	17
8.3	Krzywe ROC na danych walidacyjnych	18

1 Cel i zakres zadania

Celem niniejszego zadania była analiza porównawcza trzech popularnych architektur głębokich sieci neuronowych (ResNet50, EfficientNetB0, MobileNetV2) w zadaniu klasyfikacji obrazów.

Zakres prac obejmował:

- Przeprowadzenie 27 przebiegów uczących ($3 \text{ modele} \times 9 \text{ wariantów podziału danych}$).
- Zbadanie wpływu proporcji zbioru treningowego do testowego na jakość predykcji.
- Zbadanie zjawiska przeuczenia dla każdego z powyższych podziałów na zbiorze walidacyjnym.
- Ocenę zdolności modeli do separacji klas (za pomocą krzywych ROC).

2 Opis środowiska i narzędzi

Eksperymenty zostały przeprowadzone z użyciem aplikacji napisanej w języku Python z wykorzystaniem następujących bibliotek:

- **TensorFlow/Keras:** Implementacja modeli oraz proces uczenia.
- **Scikit-learn:** Wyliczenie miar jakości (F1-score, AUC, Accuracy).
- **Pandas/NumPy:** Zarządzanie danymi i wynikami w formacie CSV.
- **Matplotlib:** Generowanie wykresów ROC oraz analizy wyników.

Aby umożliwić uczenie modeli na rdzaniach karty graficznej, program został dostosowany do uruchamiania w kontenerze Docker. Modele były trenowane z wykorzystaniem wag startowych (Transfer Learning), a ich stany końcowe zapisywano w postaci plików wag (`.weights.h5`).

3 Organizacja badań

Badania dla każdego z trzech modeli zostały przeprowadzone z tymi samymi parametrami (Tabela 1).

4 Opis zbioru danych i przygotowanie

Zbiór danych został poddany procesowi normalizacji i augmentacji (jeśli dotyczy). Kluczowym elementem badania była zmiana wielkości zbioru treningowego. Przy skrajnym podziale (95/5), zbiór testowy liczył 2500 próbek, co pozwoliło na rzetelną, choć obarczoną pewnym szumem, estymację błędów. Dane wejściowe były skalowane zgodnie z wymaganiami konkretnych architektur (np. `preprocess_input` dla ResNet).

Tabela 1: Parametry konfiguracji eksperymentów

Parametr	Wartość
Liczba epok (<i>epochs</i>)	4
Rozmiar paczki (<i>batch_size</i>)	64
Rozmiar obrazu (<i>image_size</i>)	160×160
Współczynnik uczenia (<i>learning_rate</i>)	0.001
Wagi początkowe (<i>weights</i>)	ImageNet
Ziarno losowości (<i>seed</i>)	42
Cierpliwość (<i>patience</i>)	2
Podział walidacyjny (<i>val_ratio</i>)	0.1
Podziały standardowe	0.5, 0.7, 0.8, 0.85
Podziały ekstremalne	0.05, 0.1, 0.2, 0.9, 0.95

5 Opis modeli

W badaniu wykorzystano trzy różne sieci splotowe:

- **ResNet50:** Wykorzystuje połączenia rezydualne (skip connections), co pozwala na efektywne trenowanie głębokich sieci.
- **EfficientNetB0:** Optymalizowany pod kątem skalowania szerokości, głębokości i rozdzielczości przy użyciu współczynników złożonych.
- **MobileNetV2:** Lekka architektura dedykowana dla urządzeń mobilnych, oparta na odwróconych blokach rezydualnych i splotach separowalnych.

6 Miary oceny jakości modeli

Do oceny modeli wykorzystano następujące metryki:

6.1 Oznaczenia

:

- Miara ta określa bezwzględny stosunek poprawnych werdyktów klasyfikatora do całkowitej liczby badanych obiektów w zbiorze testowym. Wyznacza się ją globalnie dla całego procesu klasyfikacji.

$$Accuracy = \frac{\sum_c T_c}{|S_{test}|} \quad (1)$$

Zakres wartości: $[0, 1]$

Interpretacja i przykład: Przykładowa wartość $Accuracy = 0.82$ oznacza, że w 82 na 100 przebadanych przypadków ze zbioru testowego, klasyfikator podjął właściwą decyzję, natomiast pozostałe 18 uległo zamianie (np. tekst zdjęcie z koniem, zostało przypisane do klasy *frog*).

- **Macro F1-Score:** Miara ta oblicza średnią arytmetyczną z wartości F1-Score wyznaczonych niezależnie dla każdej z klas. Dzięki temu każda kategoria, niezależnie od liczby reprezentujących ją próbek, ma taki sam wpływ na wynik końcowy, co czyni ją kluczową metryką do oceny jakości modeli na niezbilansowanych zbiorach danych.

$$Macro\ F1 = \frac{1}{|C|} \sum_{c=1}^{|C|} F1_c = \frac{1}{|C|} \sum_{c=1}^{|C|} 2 \cdot \frac{Precision_c \cdot Recall_c}{Precision_c + Recall_c} \quad (2)$$

gdzie $|C|$ oznacza całkowitą liczbę klas w zbiorze.

Zakres wartości: $[0, 1]$

Interpretacja i przykład: Przykładowa wartość $Macro\ F1 = 0.899$ oznacza bardzo dobrą, uśrednioną zdolność modelu do poprawnego rozpoznawania każdej z klas. W przeciwieństwie do metryki Accuracy, wynik ten jest odporny na zdominowanie przez klasę większościową. Przykładowo: jeśli 90% obiektów w teście to klasa *dog*, a 10% to klasa *horse*, model zawsze wskazujący klasę *dog* osiągnie wysokie $Accuracy = 0.90$, ale jego $Macro\ F1$ wyniesie jedynie około 0.5 (ponieważ F1 dla klasy *horse* wynosi 0). Wysokie Macro F1 pokazuje, że klasyfikator dobrze radzi sobie z mniejszościowymi kategoriami.

- **ROC AUC (Micro/Macro OVR):** Pole pod krzywą charakterystyki roboczej odbiornika. Miara określa zdolność modelu do poprawnej separacji klas. W klasyfikacji wieloklasowej stosuje się strategię One-vs-Rest (OVR) – każda klasa przeciwko wszystkim pozostałym. Wariant Macro jest średnią arytmetyczną z tych pól, natomiast Micro agreguje wszystkie decyzje globalnie.

$$ROC\ AUC_{Macro} = \frac{1}{|C|} \sum_{c=1}^{|C|} AUC_c = \frac{1}{|C|} \sum_{c=1}^{|C|} \int_0^1 TPR_c(FPR_c) d(FPR_c) \quad (3)$$

gdzie TPR_c to czułość (True Positive Rate), a FPR_c to odsetek fałszywie pozytywnych (False Positive Rate) dla klasy c , wyznaczane dla wszystkich możliwych progów decyzyjnych.

Zakres wartości: $[0, 1]$ (w praktyce od 0.5 dla modelu losowego do 1.0 dla modelu idealnego)

Interpretacja i przykład: Przykładowa wartość $ROC\ AUC = 0.994$ oznacza niemal idealną separację klas. W kontekście badanych modeli (gdzie np. $Accuracy \approx 0.89$, a $AUC \approx 0.99$), tak wysoki wynik informuje nas, że model doskonale ocenia prawdopodobieństwa. Błędy widoczne w metrykach takich jak F1 czy Accuracy nie wynikają z tego, że model jest „zagubiony”, lecz z faktu, że dla trudnych przypadków przypisuje bardzo zbliżone, wysokie prawdopodobieństwa dwóm podobnym do siebie klasom. Prawidłowa odpowiedź zazwyczaj znajduje się na drugim miejscu w rankingu predykcji.

7 Wyniki

W niniejszym rozdziale przedstawiono szczegółowe zestawienie wyników uzyskanych przez badane architektury sieci neuronowych (ResNet50, EfficientNetB0 oraz MobileNetV2) w zależności od proporcji podziału zbioru danych na zbiór treningowy, walidacyjny i testowy.

7.1 Najlepsze wyniki dla każdego z modeli

W tabeli 2 zaprezentowano konfiguracje, dla których poszczególne modele osiągnęły najwyższą skuteczność (ang. *accuracy*) oraz najlepsze wartości uśrednionych metryk precyzji, czułości, F1-score i obszaru pod krzywą ROC (AUC).

Tabela 2: Zestawienie najlepszych wyników klasyfikacji dla badanych modeli

Model	Split Train	Split Test	Train Samples	Val Samples	Test Samples	Epochs	Acc	Macro Prec	Macro Recall	Macro F1	ROC AUC Macro OVR	ROC AUC Micro
ResNet50	0.90	0.10	40 500	4 500	5 000	4	0.8990	0.9004	0.8990	0.8993	0.9943	0.9948
EfficientNetB0	0.95	0.05	42 750	4 750	2 500	4	0.8636	0.8651	0.8636	0.8636	0.9900	0.9911
MobileNetV2	0.90	0.10	40 500	4 500	5 000	4	0.8556	0.8561	0.8556	0.8547	0.9893	0.9903

7.2 Najsłabsze wyniki dla każdego z modeli

Tabela 3 przedstawia warianty o najniższych wskaźnikach wydajnościowych, zaobserwowane przy drastycznym ograniczeniu liczby próbek treningowych (zaledwie 5% całego zbioru danych).

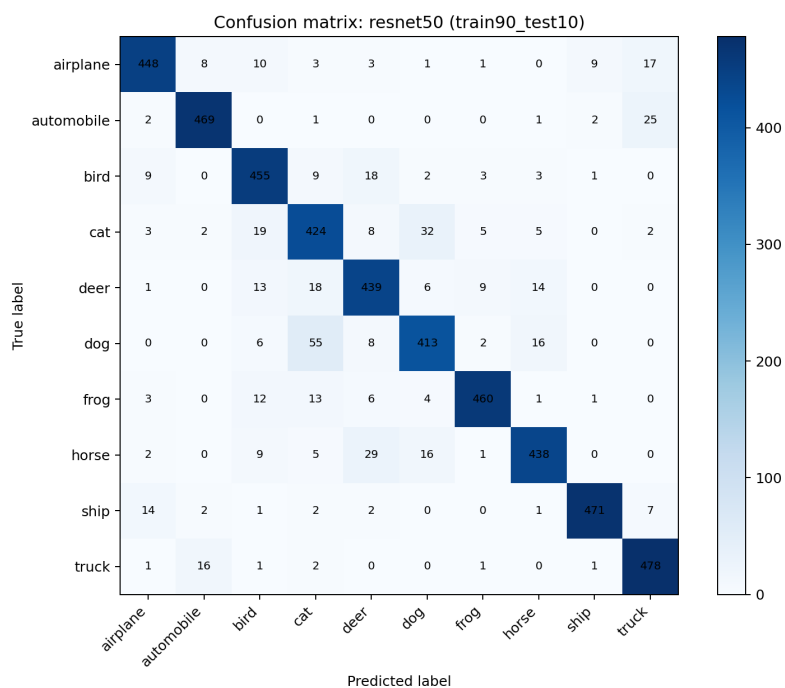
Tabela 3: Zestawienie najslabszych wyników klasyfikacji dla badanych modeli

Model	Split Train	Split Test	Train Samples	Val Samples	Test Samples	Epochs	Acc	Macro Prec	Macro Recall	Macro F1	ROC AUC Macro OVR	ROC AUC Micro
ResNet50	0.05	0.95	2 250	250	47 500	4	0.8424	0.8468	0.8424	0.8423	0.9869	0.9872
EfficientNetB0	0.05	0.95	2 250	250	47 500	4	0.7801	0.7838	0.7801	0.7802	0.9749	0.9760
MobileNetV2	0.05	0.95	2 250	250	47 500	4	0.7782	0.7895	0.7782	0.7786	0.9761	0.9755

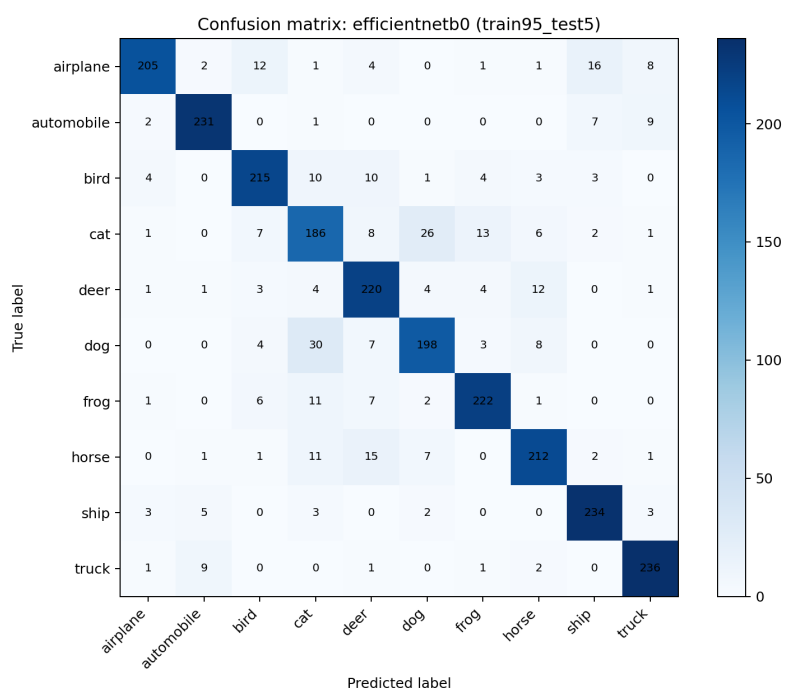
Uwaga: Wszystkie pozostałe wyniki badanych wariantów podziału zbioru, wraz z kompletem szczegółowych danych liczbowych, zostały zamieszczone w załączniku ??.

7.3 Macierze pomyłek

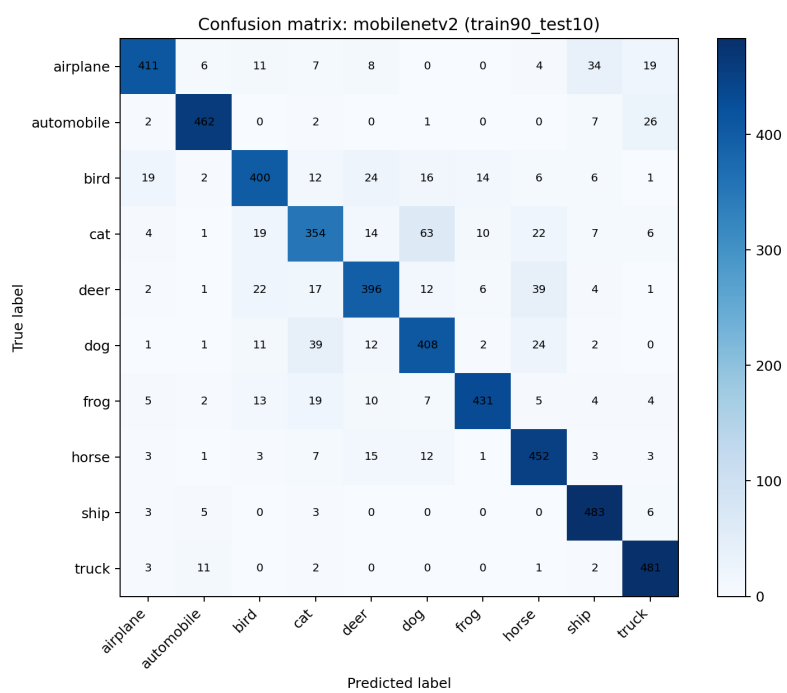
W celu głębszej analizy popełnianych błędów oraz zidentyfikowania klas najczęściej mylonych przez algorytmy, wygenerowano macierze pomyłek. Poniżej zestawiono porównanie rozkładu predykcji dla najlepszych oraz najslabszych wariantów modeli.



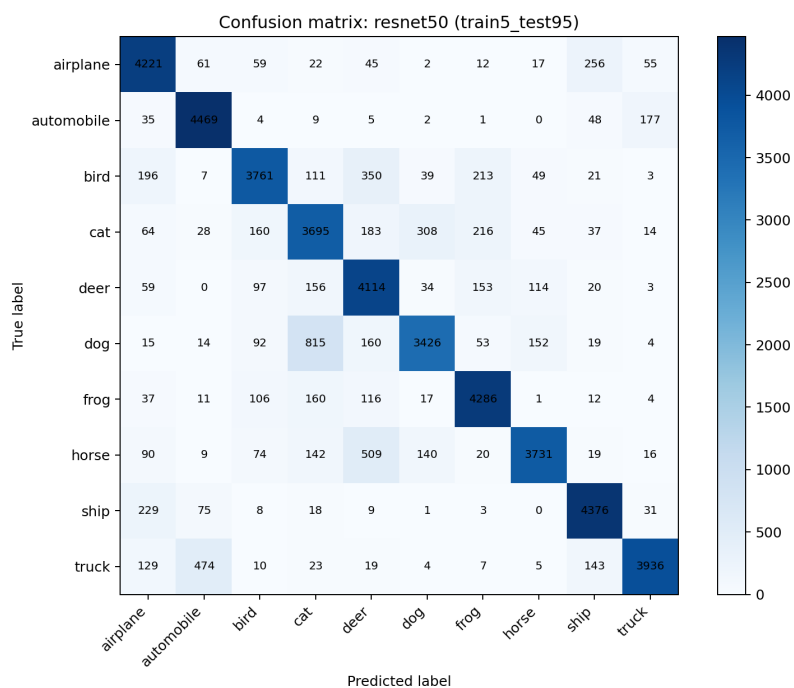
Rysunek 1: Macierz pomyłek dla najlepszej konfiguracji modelu ResNet50.



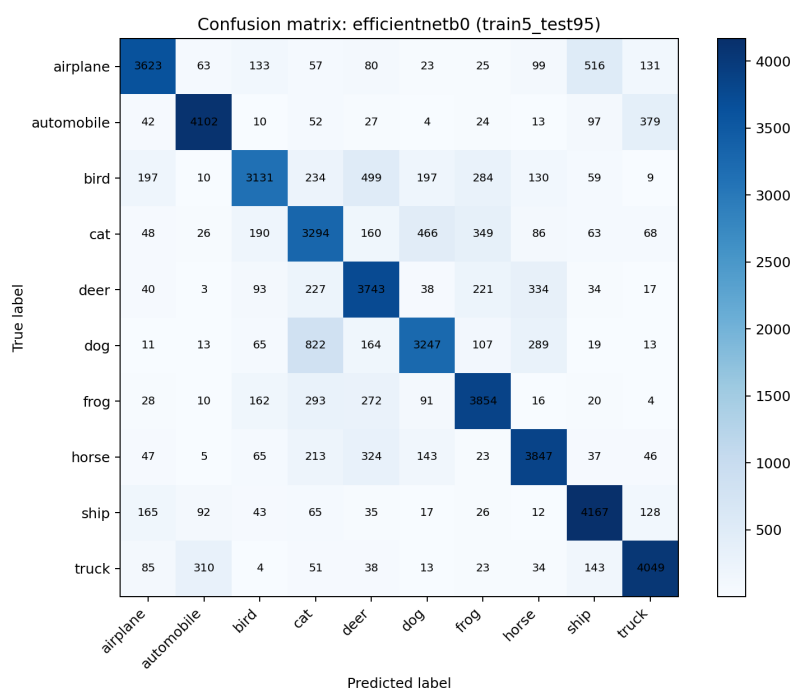
Rysunek 2: Macierz pomyłek dla najlepszej konfiguracji modelu EfficientNetB0.



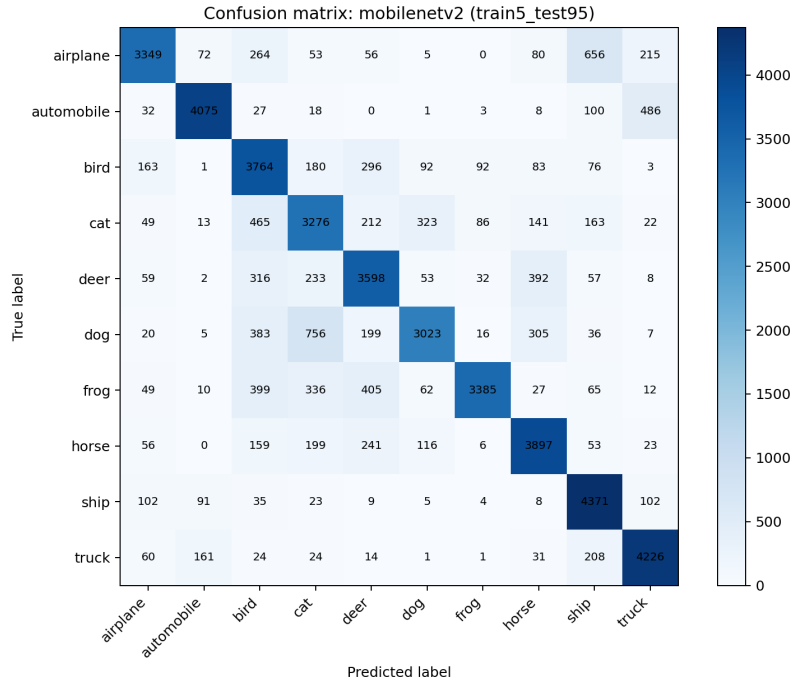
Rysunek 3: Macierz pomyłek dla najlepszej konfiguracji modelu MobileNetV2.



Rysunek 4: Macierz pomyłek dla najslabszej konfiguracji modelu ResNet50.



Rysunek 5: Macierz pomyłek dla najslabszej konfiguracji modelu EfficientNetB0.



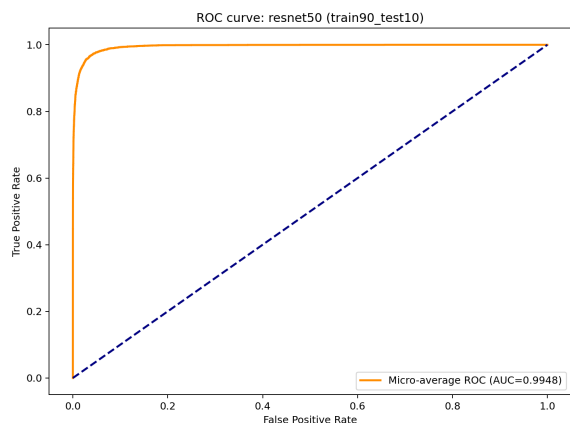
Rysunek 6: Macierz pomyłek dla najslabszej konfiguracji modelu MobileNetV2.

7.4 Analiza krzywych ROC i wartości AUC

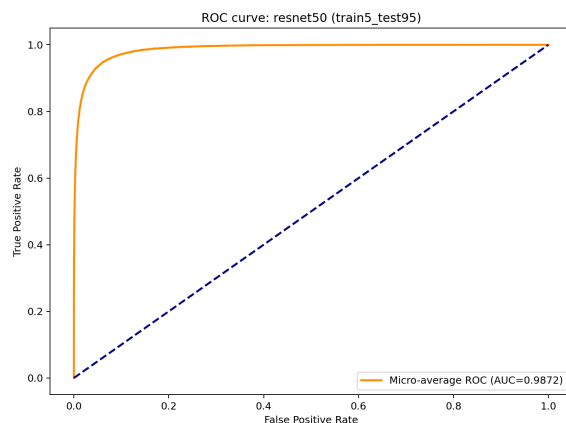
Istotnym kryterium oceny modeli wieloklasowych jest charakterystyka krzywej ROC (ang. *Receiver Operating Characteristic*) oraz pole pod tą krzywą (AUC). Obliczenia przeprowadzono z wykorzystaniem podejścia Macro OVR (*One-vs-Rest*) uśredniającego wagę każdej z klas w równym stopniu, oraz podejścia Micro, agregującego globalnie wkłady wszystkich próbek.

Jak wykazano w tabeli 2, architektura ResNet50 w optymalnej konfiguracji osiągnęła wynik ROC AUC_{Macro} = 0.9943, co wskazuje na niemal perfekcyjną zdolność do separacji klas.

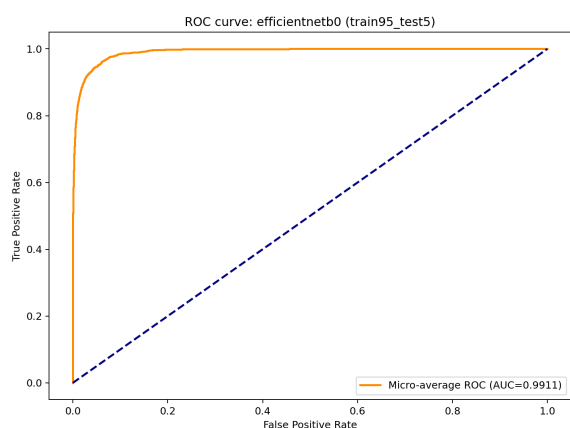
W celu zobrazowania wpływu rozmiaru zbioru treningowego na pewność predykcji, na rysunku 7 zestawiono optymalne krzywe ROC dla wszystkich trzech badanych architektur oraz, dla kontrastu, krzywą uzyskaną przez model ResNet50 trenowany na zaledwie 5% danych.



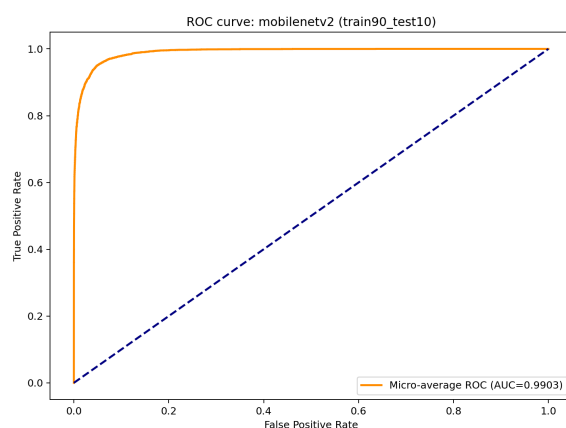
(a) ResNet50 (Najlepszy: train90_test10)



(b) ResNet50 (Najslabszy: train5_test95)



(c) EfficientNetB0 (Najlepszy: train95_test5)



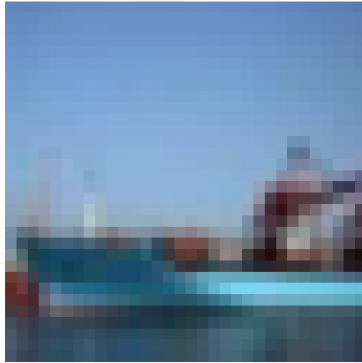
(d) MobileNetV2 (Najlepszy: train90_test10)

Rysunek 7: Zestawienie krzywych ROC oraz wartości AUC dla optymalnych konfiguracji badanych modeli oraz wariantu ze skrajnie ograniczonym zbiorem treningowym.

7.5 Wizualizacja predykcji dla najlepszych konfiguracji

Dla zobrazowania praktycznego działania wyuczonych sieci, na poniższych rysunkach przedstawiono przykładowe predykcje obrazów ze zbioru testowego dla najlepszych konfiguracji każdego z badanych modeli. Na każdy kafelek nałożono etykietę rzeczywistą (T: *True*) oraz etykietę przewidywaną przez model (P: *Predicted*). Kolorem zielonym oznaczono poprawne klasyfikacje, natomiast kolorem czerwonym przypadki błędne.

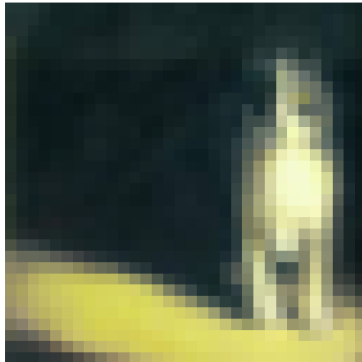
T: ship
P: ship



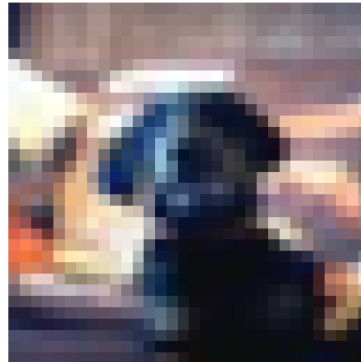
T: airplane
P: airplane



T: cat
P: cat



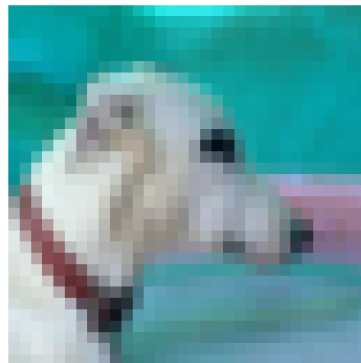
T: dog
P: dog



T: bird
P: bird



T: dog
P: dog



T: frog
P: frog



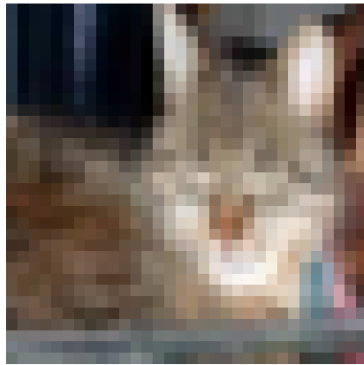
T: cat
P: cat



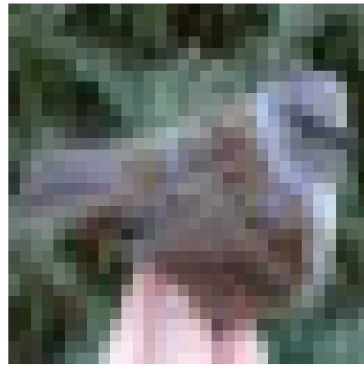


Rysunek 8: Przykładowa siatka predykcji dla najlepszej konfiguracji modelu **ResNet50** (wariant train90_test10).

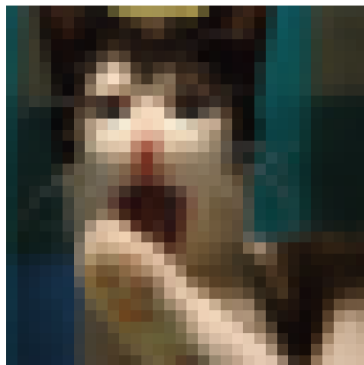
T: cat
P: cat



T: bird
P: bird



T: cat
P: cat



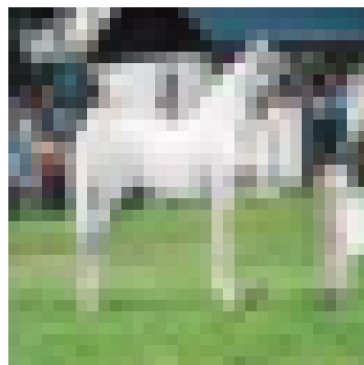
T: dog
P: dog



T: frog
P: deer



T: horse
P: horse

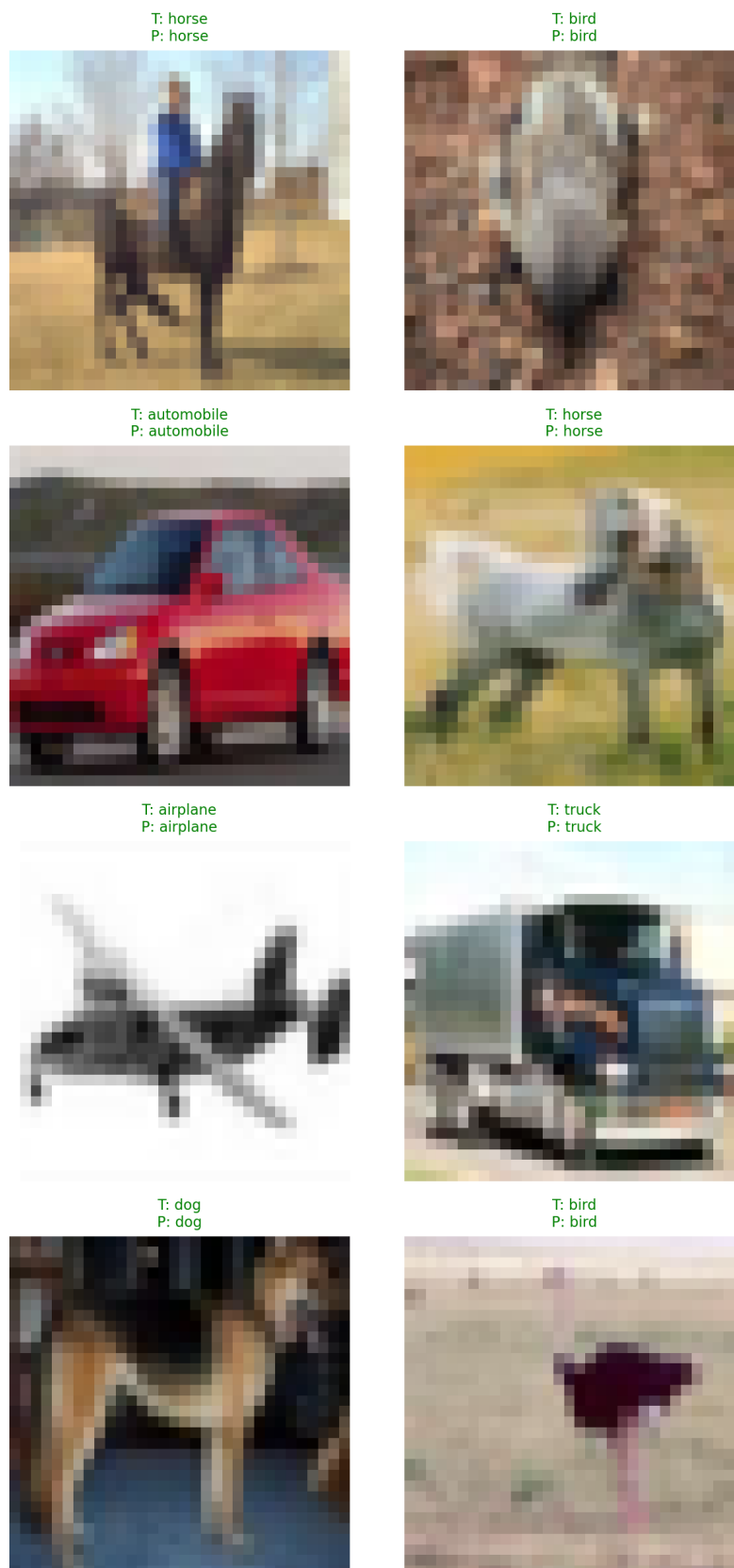


T: deer
P: deer



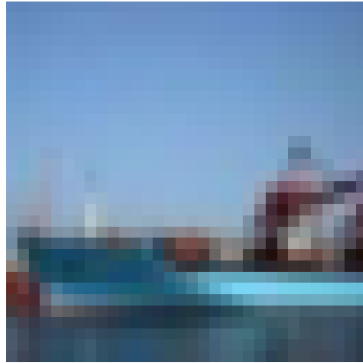
T: ship
P: ship



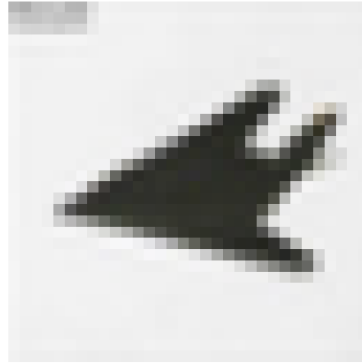


Rysunek 9: Przykładowa siatka predykcji dla najlepszej konfiguracji modelu **Efficient-NetB0** (wariant train95_test5).

T: ship
P: ship



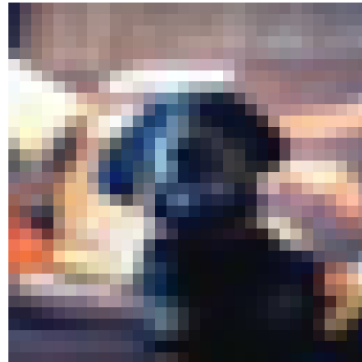
T: airplane
P: airplane



T: cat
P: dog



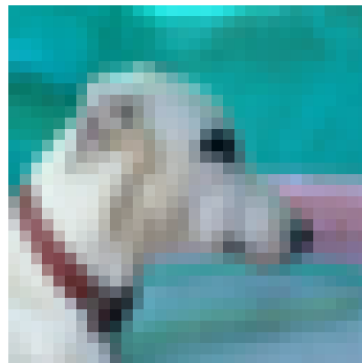
T: dog
P: dog



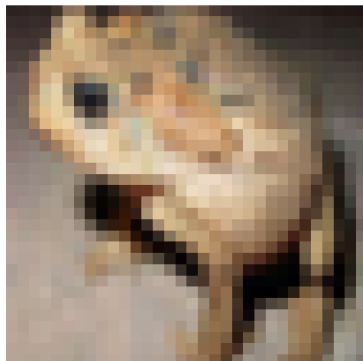
T: bird
P: bird



T: dog
P: horse



T: frog
P: frog



T: cat
P: cat





Rysunek 10: Przykładowa siatka predykcji dla najlepszej konfiguracji modelu **Mobile-NetV2** (wariant train90_test10).

8 Walidacja modeli i analiza przeuczenia

Weryfikacja zdolności uogólniania wiedzy przez badane modele została przeprowadzona na niezależnym zbiorze walidacyjnym. Pozwala to na rzetelną ocenę, czy sieci neuronowe nie uległy zjawisku przeuczenia (ang. *overfitting*) i potrafią poprawnie klasyfikować próbki, z którymi nie miały styczności podczas procesu optymalizacji wag.

8.1 Analiza wyników na zbiorze walidacyjnym

W tabeli 4 zestawiono szczegółowe wyniki skuteczności (ang. *accuracy*) oraz strat (ang. *loss*) uzyskane przez modele na zbiorach treningowym i walidacyjnym w zależności od wariantu podziału danych. Dodatkowo wyznaczono różnice tych metryk (Δ Acc oraz Δ Loss), które stanowią bezpośredni wskaźnik stopnia dopasowania modelu.

Tabela 4: Zestawienie wyników na zbiorze walidacyjnym oraz wskaźników przeuczenia modeli

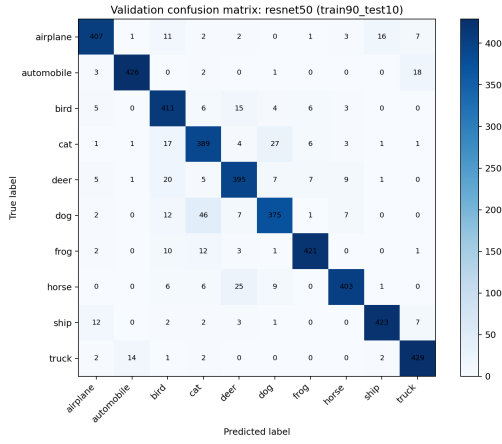
Model i wariant	Train Acc	Val Acc	Δ Acc	Train Loss	Val Loss	Δ Loss
ResNet50						
train90_test10 (Najlepszy)	0.9634	0.8936	-0.0698	0.1068	0.3524	+0.2456
train5_test95 (Najslabszy)	0.9924	0.8240	-0.1684	0.0336	0.5840	+0.5504
EfficientNetB0						
train95_test5 (Najlepszy)	0.9392	0.8653	-0.0739	0.1740	0.4287	+0.2547
train5_test95 (Najslabszy)	0.9902	0.7600	-0.2302	0.0333	0.8520	+0.8187
MobileNetV2						
train90_test10 (Najlepszy)	0.9304	0.8540	-0.0764	0.2016	0.4727	+0.2711
train5_test95 (Najslabszy)	0.9889	0.7480	-0.2409	0.0401	0.8687	+0.8286

Analiza danych wskazuje na silną zależność między rozmiarem zbioru treningowego a stabilnością modelu. W przypadkach skrajnych (wariant `train5_test95`), wszystkie modele osiągają na zbiorze treningowym skuteczność rzędu 99%, jednak na zbiorze walidacyjnym wartość ta drastycznie spada (spadek o blisko 24 punkty procentowe dla MobileNetV2 oraz wzrost straty walidacyjnej o ponad 0.8). Świadczy to o zapamiętaniu specyfiki nielicznych danych treningowych zamiast wyuczenia ogólnych cech klas.

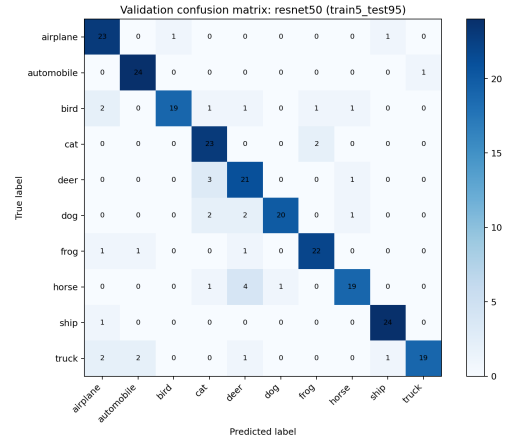
Z kolei dla optymalnych podziałów (np. ResNet50 w wariantcie `train90_test10`), spadek skuteczności walidacyjnej wynosi zaledwie $\approx 7\%$, co potwierdza wysoką stabilność i poprawną generalizację.

8.2 Macierze pomyłek dla zbioru walidacyjnego

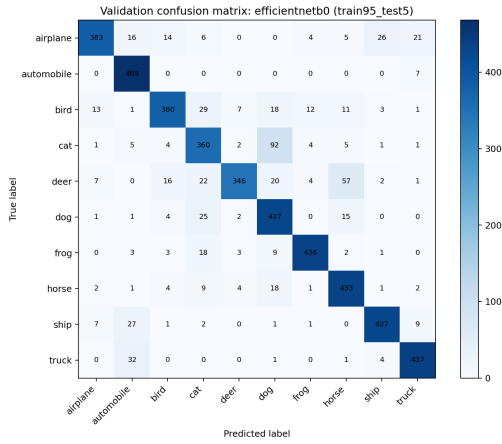
Aby zweryfikować, w jaki sposób rozkładają się błędy predykcji na danych walidacyjnych, wygenerowano odpowiadające im macierze pomyłek. Zgodnie z przyjętą metodyką, na rysunku 11 zaprezentowano zestawienie dla najlepszego i najslabszego wariantu sieci ResNet50 oraz optymalnych konfiguracji pozostałych architektur.



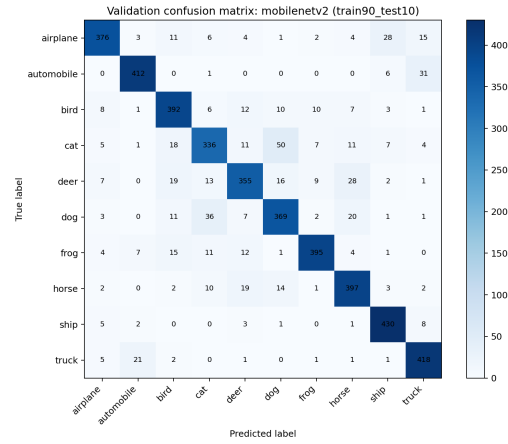
(a) ResNet50 (Najlepszy: train90_test10)



(b) ResNet50 (Najslabszy: train5_test95)



(c) EfficientNetB0 (Najlepszy: train95_test5)

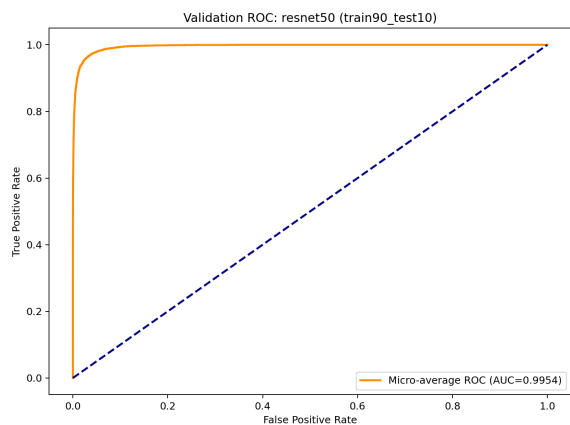


(d) MobileNetV2 (Najlepszy: train90_test10)

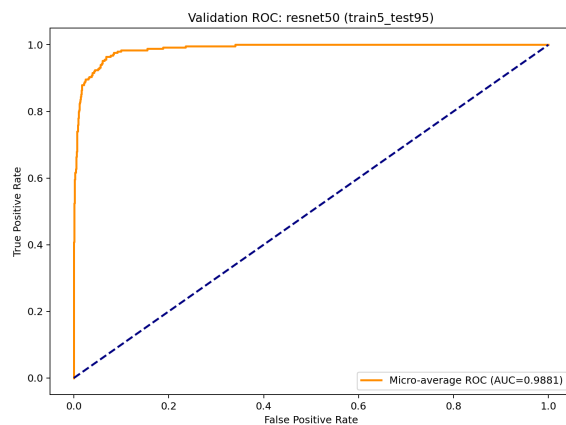
Rysunek 11: Macierze pomyłek uzyskane na zbiorze walidacyjnym dla wybranych konfiguracji badanych modeli.

8.3 Krzywe ROC na danych walidacyjnych

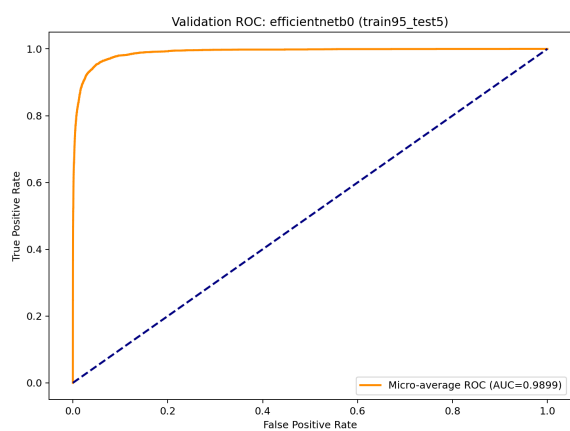
Uzupełnieniem oceny walidacyjnej jest analiza zdolności rozdzielczej klasyfikatorów przedstawiona za pomocą krzywych ROC oraz metryki AUC. Rysunek 12 przedstawia wykresy wyznaczone na próbkach ze zbioru walidacyjnego. Warto zauważyć, że pomimo spadku ogólnej dokładności (Acc) w wariantach z małą ilością danych, pole pod krzywą (ROC AUC) na zbiorze walidacyjnym wciąż utrzymuje się na stosunkowo wysokim poziomie, co sugeruje, że modele zachowują pewną zdolność do rangowania prawdopodobieństw poszczególnych klas.



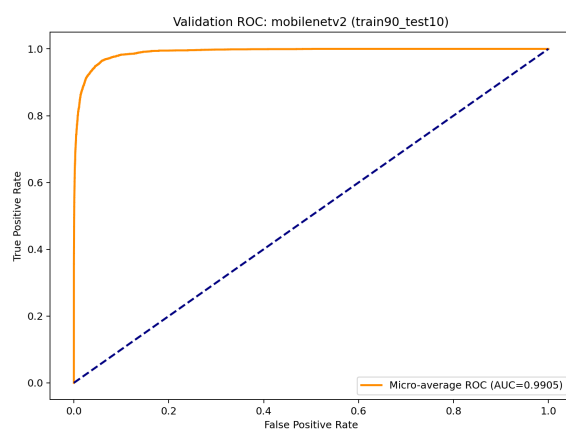
(a) ResNet50 (Najlepszy: train90_test10)



(b) ResNet50 (Najslabszy: train5_test95)



(c) EfficientNetB0 (Najlepszy: train95_test5)



(d) MobileNetV2 (Najlepszy: train90_test10)

Rysunek 12: Krzywe ROC oraz wartości AUC wyznaczone na zbiorze walidacyjnym dla kluczowych wariantów badanych architektur.